

Gliwice, 09.01.2025

Programowanie I
projekt zaliczeniowy
"Warcaby"

Kamil Kowalczyk gr. lab. 1.1

1. Opis projektu.

Projekt „Warcaby” to gra dla 2 graczy, gdzie jeden gra pionami białymi, a drugi pionami czarnymi. Celem jest zabicie wszystkich pionów przeciwnika. Gra jest przeznaczona na system Windows 10 i Windows 11.

2. Funkcjonalności.

- Rozgrywka dla 2 graczy
- Proste menu
- Przycisk wyjścia z gry
- Przycisk rozpoczęcia nowej rozgrywki
- Przycisk wyświetlający informacje o programie
- Wsparcie dla rozdzielczości (**ze skalowaniem ustawionym na 100%**):
1024 x 768, 1152 x 864, 1280 x 720, 1280 x 768, 1280 x 800, 1280 x 960,
1280 x 1024, 1360 x 768, 1366 x 768, 1400 x 1050, 1440 x 900,
1600 x 900, 1680 x 1050, 1920 x 1080, 2560 x 1440, 3840 x 2160

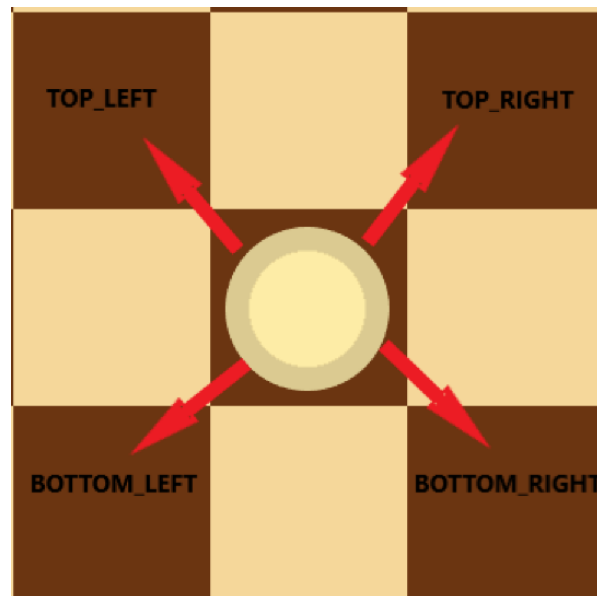
3. Przebieg realizacji.

1. Stworzenie zamysłu wyglądu aplikacji, rozłożenia elementów UI.
2. Stworzenie grafik dla planszy do gry, pionków oraz ich odpowiedników po awansie do królowej
3. Stworzenie podstawowych klas do obsługi logiki gry, takich jak: **Field**, **Pawn**, **PawnMove**
4. Stworzenie klasy **App**, która będzie wyświetlać cały wygląd gry.
5. Stworzenie układu oraz wyglądu aplikacji w klasie **MainFrame** (utworzenie planszy w oknie aplikacji, miejsca na teksty, guziki np. **Info**, **Quit** oraz dostosowanie kolorystyki)
6. Stworzenie kodu generującego pola planszy, pionki w klasie **Game**, gdzie znajduje się logika gry.
7. Utworzenie kodu w klasie **MainFrame**, który tworzył przyciski, które są pionkami na planszy
8. Stworzenie typu wyliczeniowego **Direction** oraz kilku funkcji do niego, żeby móc rozpocząć tworzenie algorytmu szukającego dostępnych ruchów.
9. Stworzenie algorytmu szukającego dostępnych ruchów (więcej o nim w dalszej części) w klasie **Game**.
10. Stworzenie obsługi przemieszczenia pionka na planszy oraz bicia innych pionków (zarówno logiki w klasie **Game** jak i wyglądu w klasie **MainFrame**).
11. Dodanie powiadomień w klasie **MainFrame**, jeżeli pionek nie może wykonać ruchu (ze względu na brak możliwości poruszenia się, w momencie, kiedy dostępne jest bicie lub w momencie, kiedy aktualny ruch należy do innego gracza).

12. Stworzenie kodu aktualizującego teksty w lewym panelu aplikacji oraz dodanie obsługi przycisków: **Quit**, **Info** w klasie **MainFrame**.
13. Dodanie obsługi przycisku **New game** w klasie **MainFrame** oraz stworzenie metody w klasie **Game** do rozpoczynania nowej rozgrywki.

Pliki projektu:

- **App.h, App.cpp** – służy do uruchomienia aplikacji oraz do wyświetlenia jej okna
- **MainFrame.h, MainFrame.cpp** – zawiera graficzną część aplikacji, obsługę kliknięć, aktualizacje wyświetlanej treści
- **Game.h, Game.cpp** – zawiera logiczną część aplikacji, generuje pionki, sprawdza dostępne ruchy i zbite pionki itp.
- **Direction.h, Direction.cpp** – zawiera typ wyliczeniowy **Direction**, który wskazuje na kierunek poruszenia się, i zawiera 2 funkcje, które wykonują pewne operacje związane z tym typem wyliczeniowym (zwracanie kierunku przeciwnego, zwiększanie bądź zmniejszanie koordynatów rzędu oraz kolumny w zależności od kierunku)



- **Field.h, Field.cpp** – zawiera klasę, która jest wirtualnym polem na planszy
- **ImagePanel.h, ImagePanel.cpp** – jest to plik zawierający panel, który może przyjąć obrazek jako tło. Kod pochodzi z https://wiki.wxwidgets.org/An_image_panel
- **Pawn.h, Pawn.cpp** – zawiera klasę, która jest wirtualnym pionkiem, oraz typ wyliczeniowy **Color** wskazujący na kolor biały lub czarny.
- **PawnMove.h, PawnMove.cpp** – zawiera klasę, która jest wirtualnym ruchem możliwym do zagrania, oraz typ wyliczeniowy **MoveType**, który wskazuje na typ ruchu (zwykły ruch lub ruch bijący)
- Pliki z grafikami w folderach **assets/final/x1.0**, **assets/final/x0.5**, **assets/final/x2.0**: **black_pawn.png**, **black_queen.png**, **board.png**, **white_pawn.png**, **white_queen.png** (Ikonka korony została stworzona przez **smashingstocks** - Flaticon <https://www.flaticon.com/free-icons/crown>)

Algorytm szukający dostępnych ruchów:

Algorytm szukający dostępnych ruchów opiera się na **rekurencji**. Tłumacząc go rozdzielię go na 2 przypadki. **Przypadek 1** to szukanie ruchów dla zwykłych pionów, a **przypadek 2** to szukanie ruchów dla królowych.

Przypadek 1:

Najpierw algorytm sprawdza czy zbiór pionów do zbiccia jest pusty, żeby upewnić się, czy ruch do szukania będzie zwykłym ruchem.

Potem sprawdzany jest kolor pionu, aby móc określić kierunek poruszania. Dla pionów **białych** to **TOP_LEFT** i **TOP_RIGHT**, a dla pionów **czarnych** to **BOTTOM_LEFT** i **BOTTOM_RIGHT**.

```
if (pawnsToCapture.size() == 0) {  
    /* ... */  
    if (pawn->color == WHITE) {  
        directionsWithMovementOnly.insert(TOP_LEFT);  
        directionsWithMovementOnly.insert(TOP_RIGHT);  
    }  
    else {  
        directionsWithMovementOnly.insert(BOTTOM_LEFT);  
        directionsWithMovementOnly.insert(BOTTOM_RIGHT);  
    }  
    /* ... */  
}
```

Algorytm sprawdza dla każdego z kierunków czy na polach odpowiadających kierunkowi ruchu stoją piony, czy może pola są puste.

W momencie, kiedy pole jest puste powstaje zwykły ruch pionu na to pole przez wywołanie metody **createPawnMove**.

```
for (Direction direction : directions) {  
    int destRow = startRow;  
    int destCol = startCol;  
    pawnsToCaptureLocal = pawnsToCapture;  
    incrementDestinationCoordinates(direction, destRow, destCol);  
    if (destRow >= 0 && destRow <= 7 && destCol >= 0 && destCol <= 7) {  
        Field* field = board[destRow][destCol];  
        if (field->pawn != nullptr) { ... }  
        else if (directionsWithMovementOnly.count(direction) == 1) {  
            createPawnMove(destRow, destCol, pawn, pawnMoves.size(), MOVE);  
            /* ... */  
        }  
    }  
}
```

Jeżeli zaś na sprawdzanym polu stoi pion koloru przeciwnego, algorytm sprawdza kolejne pole na planszy idąc dalej w tym samym kierunku ruchu i sprawdza czy to pole jest wolne.

```
if (field->pawn != nullptr) {
    if (field->pawn->color != whoseTurn && pawnsToCapture.count(field->pawn) == 0) {
        Pawn* pawnToCapture = field->pawn;
        incrementDestinationCoordinates(direction, destRow, destCol);
        if (destRow >= 0 && destRow <= 7 && destCol >= 0 && destCol <= 7) {
            field = board[destRow][destCol];
            if (field->pawn == nullptr) {
                pawnsToCaptureLocal.insert(pawnToCapture);
                checkForMove(destRow, destCol, pawn, getCounterDirection(direction), pawnsToCaptureLocal);
            }
        }
    }
}
```

Jeśli pole jest wolne, to algorytm dodaje pionka z pola poprzedniego do zbioru zawierającego pionki, które mogą być zbite podczas ruchu. W kolejnym kroku algorytm poprzez rekurencję wywołuje się raz jeszcze.

```
if (field->pawn == nullptr) {
    pawnsToCaptureLocal.insert(pawnToCapture);
    checkForMove(destRow, destCol, pawn, getCounterDirection(direction), pawnsToCaptureLocal);
}
```

W tym momencie przesyłane są do jego metody następujące parametry:

destRow – numer rzędu na planszy, w którym stoi pion po możliwym z biciu ostatniego pionu w zbiorze pionów możliwych do zbitia podczas ruchu

destCol – numer kolumny na planszy, w którym stoi pion po możliwym z biciu ostatniego pionu w zbiorze pionów możliwych do zbitia podczas ruchu

pawn – wskaźnik do pionu, dla którego wykonywany jest ruch

getCounterDirection(direction) – kierunek przeciwny do tego, z którego przyszedł pionek, jest to po to, żeby podczas szukania dodatkowych pionów do bicia, nie szukać ich tam, skąd przyszedł pionek wykonujący ruch

pawnsToCaptureLocal – jest to zbiór pionów, które mogą być zbite podczas ruchu

Algorytm **przestaje wywoływać** kolejne rekurencje w momencie, kiedy **warunek na poprzednim zdjęciu nie będzie spełniony**.

Ruch bijący jest tworzony dzięki warunkowi, gdzie jest porównywana ilość pionów do zbitia w zmiennej **numberOfPawnsToCaptureAtBeginning** (jest to ilość pionów do zbitia przed wywołaniem kolejnej rekurencji) z ilością pionów do zbitia z wywołanej ostatnio rekurencji. Kiedy wywołanie rekurencji nie znalazło żadnych pionów do zbitia, czyli że nie było kolejnych ruchów bijących, oznacza to, że aktualne wywołanie algorytmu (przed

wywołaniem rekurencji) zawiera ruch bijący z maksymalną liczbą pionów. Ruch bijący jest następnie tworzony przez wywołanie metody `createPawnMove`.

```
if (numberOfPawnsToCaptureAtBeginning == pawnsToCapture.size() && pawnsToCapture.size() != 0) {
    createPawnMove(startRow, startCol, pawn, pawnMoves.size(), CAPTURE, pawnsToCapture);
    isAnyCapturingMove = true;
    if (pawnsToCapture.size() > maxPawnsToCapture) {
        maxPawnsToCapture = pawnsToCapture.size();
    }
}
```

Przypadek 2:

Na samym początku, kiedy pion jest królową, jego kierunki poruszania to `TOP_LEFT`, `TOP_RIGHT`, `BOTTOM_LEFT`, `BOTTOM_RIGHT`.

```
if (pawnsToCapture.size() == 0) {
    if (pawn->isQueen) {
        directionsWithMovementOnly = { BOTTOM_LEFT, BOTTOM_RIGHT, TOP_LEFT, TOP_RIGHT };
    }
}
```

W kolejnym kroku sprawdzamy czy kierunek, z którego przyszła królowa jest różny od `NONE` (inaczej pozycja początkowa). Domyślnie wywołanie metody `checkForMove` ustawia parametr `queenDirection` na `NONE`, bo na początku królowa może poruszać się w każdym z 4 kierunków. W dalszej części znaczenie tego warunku i tego parametru zostanie dokładniej omówione.

```
if (pawnsToCapture.size() == 0) {
    if (pawn->isQueen) {
        directionsWithMovementOnly = { BOTTOM_LEFT, BOTTOM_RIGHT, TOP_LEFT, TOP_RIGHT };
    }
    else { ... }
    if (queenDirection != NONE) {
        directionsWithMovementOnly = { queenDirection };
        directions = { queenDirection };
    }
}
```

Następnie kroki dotyczące szukania ruchów bijących są takie same.

Różnica następuje natomiast przy szukaniu ruchów zwykłych, ponieważ teraz w miejscu, gdzie tworzyliśmy ruch pionu sprawdzamy też czy pion jest królową. Jeśli jest królową, to może się poruszać po **dowolnie długich przekątnych**, więc dlatego zostaje wywołana rekurencyjna metoda `checkForMove`, która będzie sprawdzać kolejne możliwe zwykłe ruchy przesuwające królową. Dlatego teraz jest uzupełniany ostatni parametr tej metody (`queenDirection`) po to, aby określić po jakim kierunku przekątnej królowa ma się poruszać.

```
if (field->pawn != nullptr) { ... }
else if (directionsWithMovementOnly.count(direction) == 1) {
    createPawnMove(destRow, destCol, pawn, pawnMoves.size(), MOVE);
    if (pawn->isQueen) {
        checkForMove(destRow, destCol, pawn, getCounterDirection(direction), pawnsToCaptureLocal, direction);
    }
}
```

Dlatego w tej części kodu sprawdzamy czy kierunek poruszania się królowej jest różny od **NONE**, żeby algorytm sprawdzał tylko i wyłącznie ruchy po konkretnej przekątnej.

```
if (pawnsToCapture.size() == 0) {  
    if (pawn->isQueen) {  
        directionsWithMovementOnly = { BOTTOM_LEFT, BOTTOM_RIGHT, TOP_LEFT, TOP_RIGHT };  
    }  
    else { ... }  
    if (queenDirection != NONE) {  
        directionsWithMovementOnly = { queenDirection };  
        directions = { queenDirection };  
    }  
}
```

Zewnętrzne biblioteki:

W aplikacji została wykorzystana zewnętrzna biblioteka **wxWidgets** służąca do tworzenia wieloplatformowych aplikacji GUI (Graphical User Interface).

W projekcie zostały wykorzystane klasy z biblioteki **wxWidgets**, takie jak: `wxString`, `wxSize`, `wxPoint`, `wxCommandEvent`, `wxDisplay`, `wxRect`, `wxFrame`, `wxBoxSizer`, `wxPanel`, `wxButton`, `wxBitmap`, `wxImage`, `wxBitmapButton`, `wxStaticText`

`wxString` - klasa stringowa oferowany przez bibliotekę

`wxSize` - rozmiar elementów GUI np. guzika, panelu, okna itd.

`wxPoint` - punkt położenia elementów GUI na np. panelu, oknie

`wxCommandEvent` - zawiera informacje o zdarzeniach np. kliknięcia

`wxDisplay` - określa informacje o wyświetlaczach podłączonych do systemu

`wxRect` - prezentuje prostokąt z koordynatami całkowitymi

`wxFrame` - okno pozwalające na zmianę jego rozmiaru

`wxBoxSizer` - pozwala na porządkowanie elementów wertykalnie lub horyzontalnie

`wxPanel` - okno, gdzie mogą być umieszczane komponenty GUI

`wxButton` - komponent GUI będący guzikiem

`wxBitmap` - zawiera funkcjonalność do łatwego i szybkiego prezentowania bitmapy zależnej od platformy

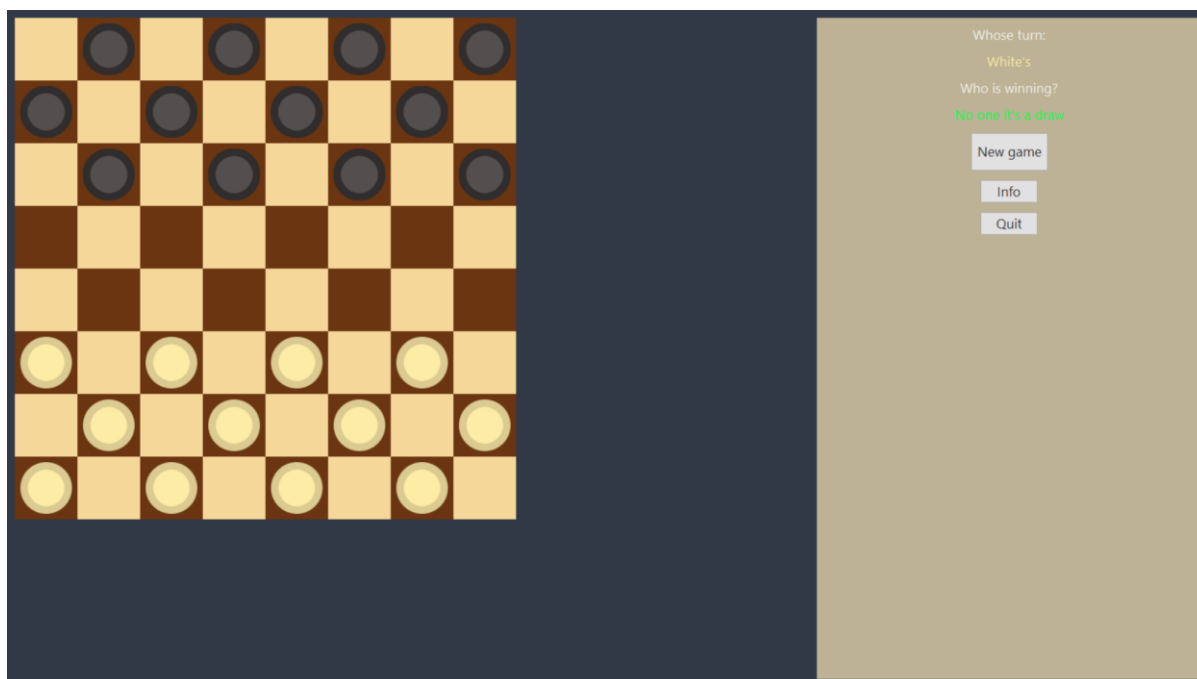
`wxImage` - zawiera funkcjonalność do tworzenia obrazów niezależnych od platformy

`wxBitmapButton` - komponent GUI będący przyciskiem, na którym jest bitmapa

`wxStaticText` - komponent GUI wyświetlający tekst

4. Instrukcja użytkownika.

Po uruchomieniu gry ukazuje się takie okno obejmujące pełny ekran:

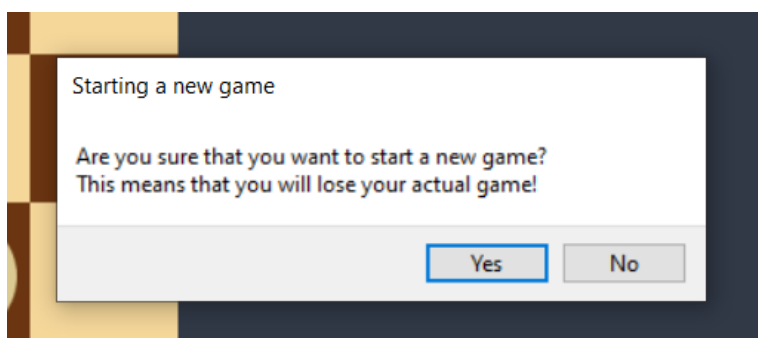


Z prawej strony znajduje się panel wyświetlający następujące informacje:

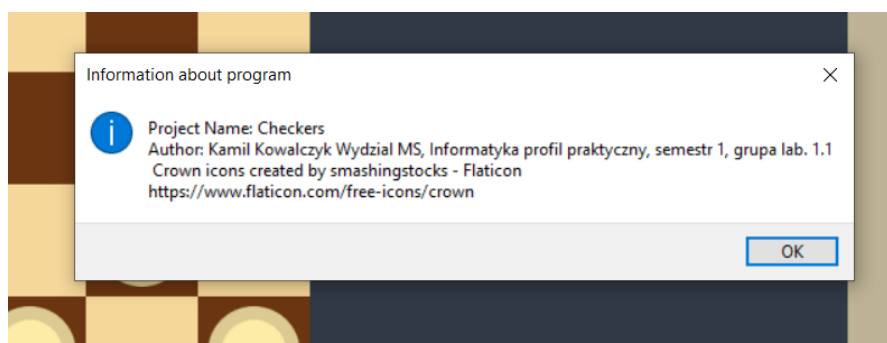
- jaki kolor pionów powinien teraz wykonać ruch
- kto wygrywa i o ile pionów wygrywa

Poniżej tych informacji znajdują się 3 przyciski:

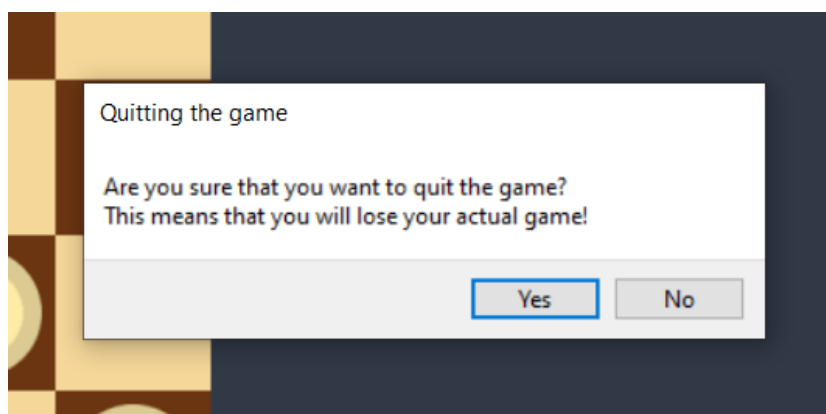
- **New game** – rozpoczyna nową grę



- **Info** – wyświetla informacje o programie

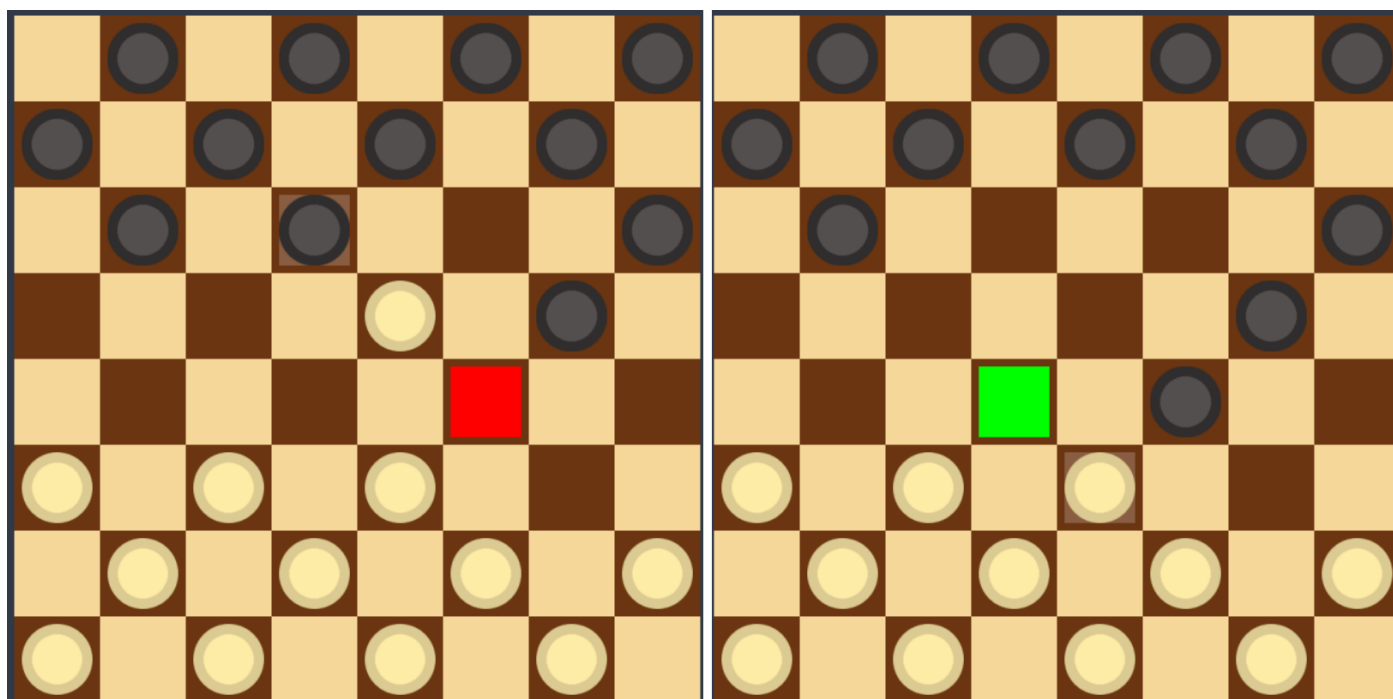


- **Quit** – wyłącza program

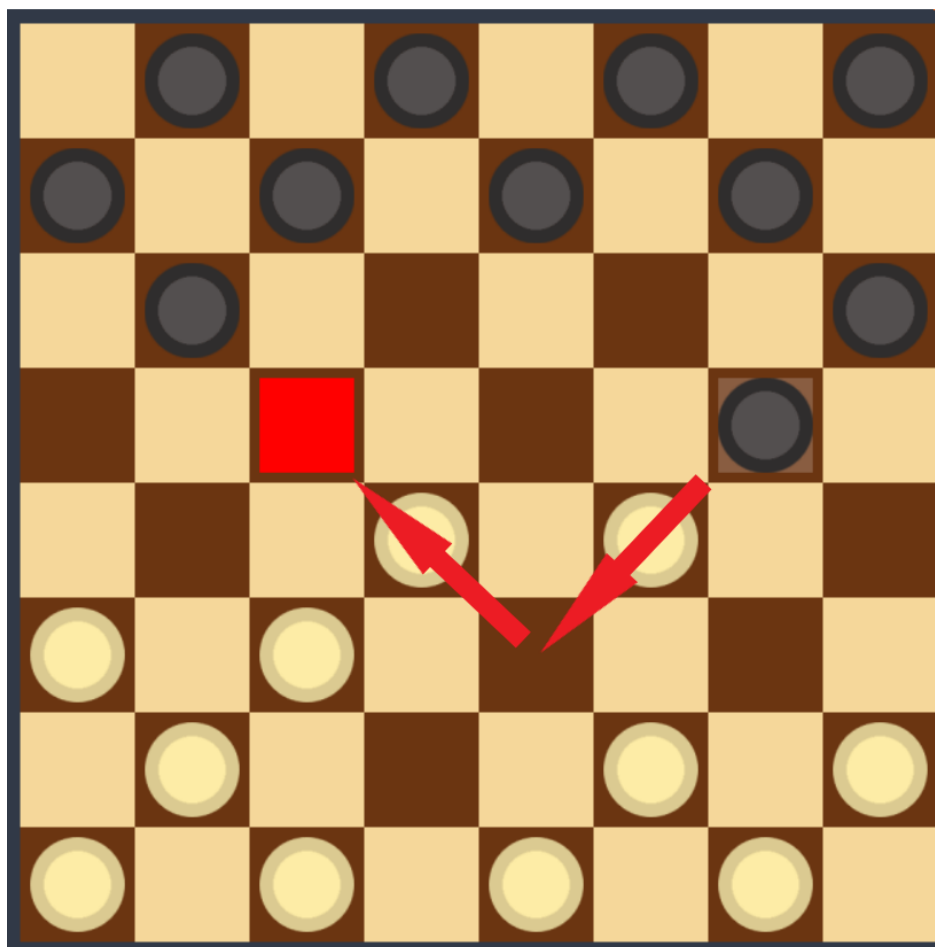


Z lewej strony znajduje się plansza do gry

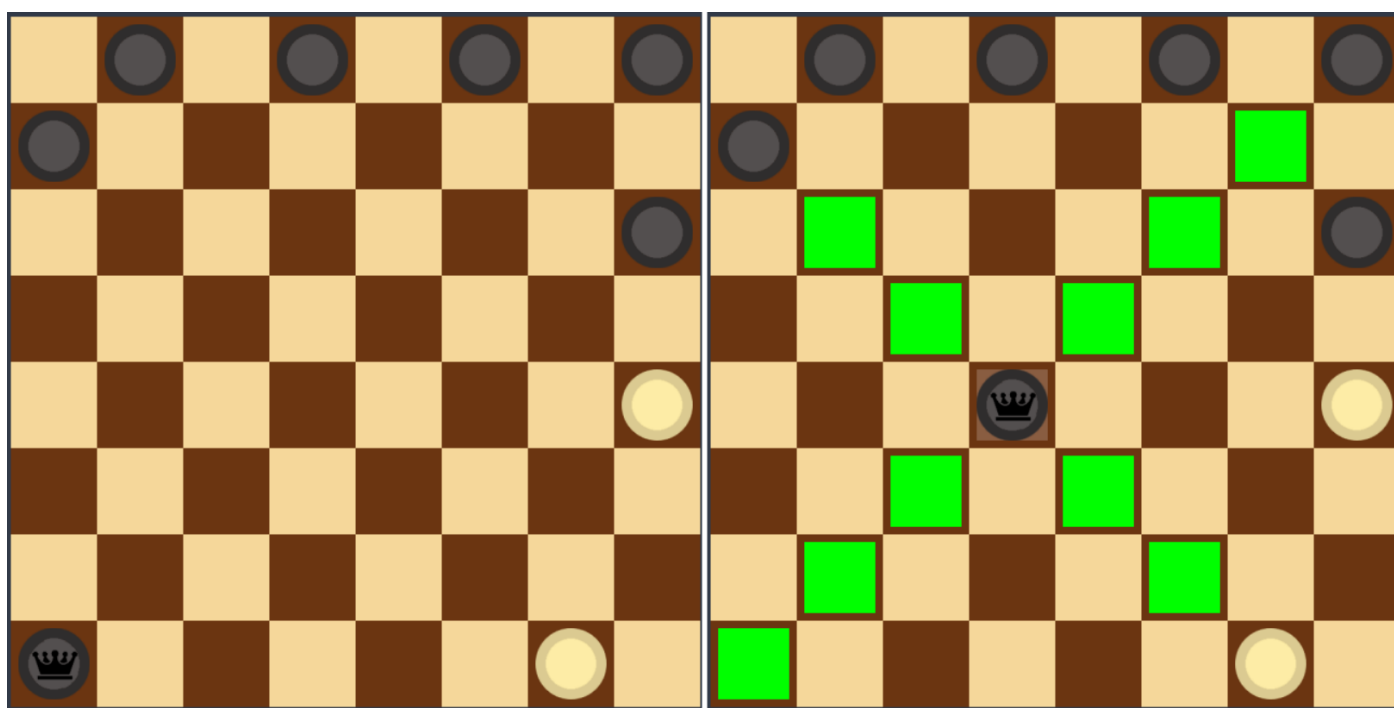
W momencie naciśnięcia pionka o kolorze gracza, który aktualnie powinien wykonać ruch, może pokazać się przycisk zielony lub czerwony. Kolor zielony to zwykłe przesunięcie pionu, a kolor czerwony to ruch bijący. Przycisk pojawia się w miejscu, gdzie będzie stał pion po dokonaniu ruchu. **UWAGA:** pionek może poruszać się tylko do przodu po przekątnej, ale może wykonać bicie po przekątnej do tyłu, jak i do przodu. Oprócz tego **gracz zawsze wykonuje ruch bijący**, który zbije jak najwięcej pionów.



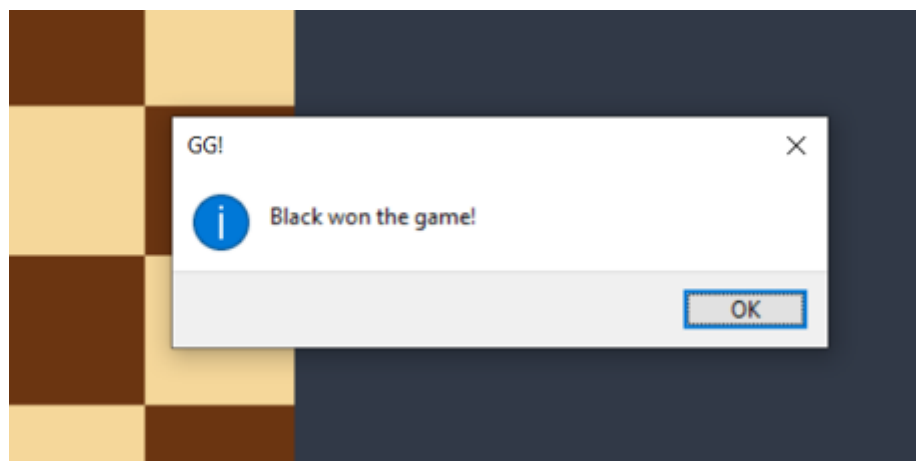
Podczas ruchu bijącego może zostać zbite więcej pionów niż tylko jeden (strzałki na grafice poniżej nie występują w aplikacji, mają one na celu łatwiejsze zrozumienie mechaniki bicia).



Pionek dochodząc na przeciwny kraniec planszy może awansować na królową

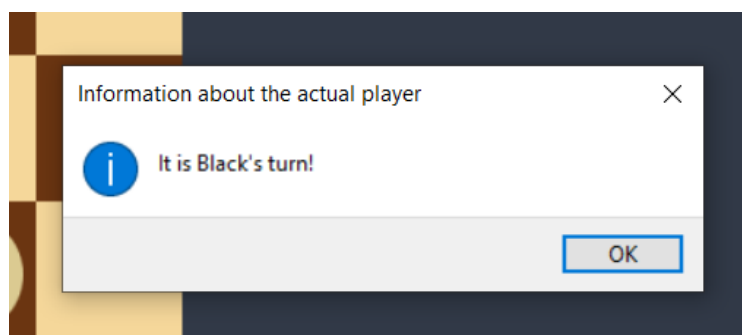


Po zбиciu wszystkich pionów przeciwnika ukazuje się odpowiedni komunikat

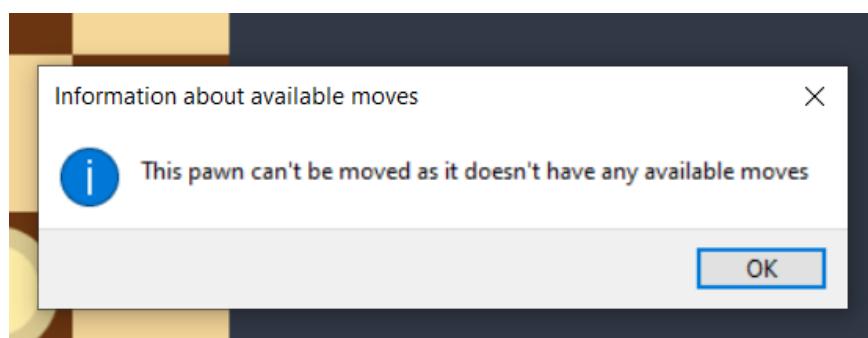


Powiadomienia dla gracza

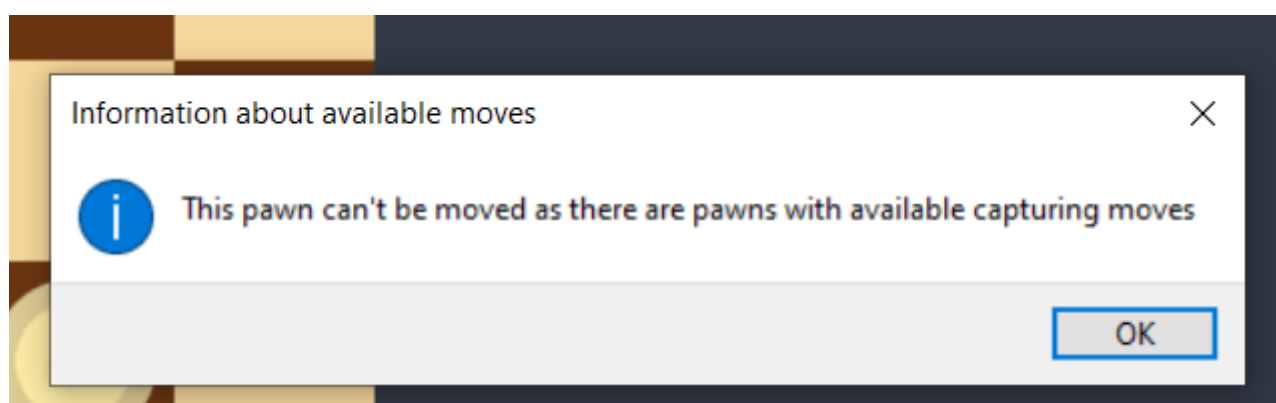
Po naciśnięciu pionu, który nie wykonuje obecnie ruchu



Po naciśnięciu pionu, który nie posiada żadnych dostępnych ruchów



Po naciśnięciu pionu w momencie, kiedy inny pion ma możliwy do zagrania ruch bijący



5. Podsumowanie i wnioski.

Udało się zrealizować:

- pełną mechanikę gry w warcaby o określonych zasadach
- proste i przejrzyste UI
- powiadomienia w momencie, kiedy gracz chce zrobić coś niedozwolonego (np. ruszyć pion, który nie posiada ruchów)

Co można by zrealizować w przyszłości:

- przeciwnika w postaci komputera
- wybór zasad, które mają obowiązywać w rozgrywce